CSE/DSC 234 Project 1 Report

Before creating the final curated run, I used the released validation set for exploratory tuning. The final curated experiment then used the stronger settings from that exploration as the starting point, while keeping `data/golden_qa.json` separate from the validation/dev set.

1. Data Creation Methodology

The submitted golden data file is `data/golden_qa.json`. It contains 25 manually curated Q&A pairs over the RapidFire AI documentation corpus. The file follows the same schema as the released validation set:

- `question_id`
- `question`
- `reference_answer`
- `source_evidence`

The questions cover factual lookup, conceptual explanation, procedural usage, API/schema behavior, and comparative reasoning. Each answer was checked against the raw `.rst` documentation line spans listed in `source_evidence`. The source audit notes are kept in `data/golden_qa_source_audit.md`.

The released validation set is treated only as a validation/dev set for tuning and evaluation, not as the submitted golden data.

2. Final Pipeline Architecture

The final runnable pipeline is implemented in `main.py`. The two concrete commands used for the final rerun were:

```
python3 main.py --input validation-set-golden-qa-pairs.json --output outputs/validation_answ
python3 main.py --input data/golden_qa.json --output outputs/curated_golden_answers.json
```

The final pipeline loads the RapidFire documentation from `sourcedocs/`, splits the `.rst` files into line-aware chunks, builds an in-memory TF-IDF index, retrieves candidate chunks, applies lexical/title/API-symbol reranking, assembles source-aware context within the 2,000-token budget, and generates an answer with an OpenAI-compatible chat model when credentials are available. If no model API is available, it uses an extractive fallback so the script still runs end to end.

Each output item contains `question_id`, `answer`, `sources`, and `retrieved_context`, matching the project statement.

3. Experimentation Methodology

The RapidFire AI experimentation layer is implemented in `rapidfire_experiments.py`. It follows the tutorial notebook structure:

- `RFLangChainRagSpec` defines document loading, chunking, OpenAI-compatible embeddings, FAISS vector store retrieval, search configuration, reranking, and document serialization.
- `RFGridSearch` expands multi-config knobs.
- `Experiment(..., mode="evals")` and `run_evals(...)` run the comparisons.

The intended RapidFire grid has 12 configurations:

Knob	Values
Chunking	`chunk_size=320, overlap=64`; `chunk_size=512, overlap=96`
Search	similarity `k=5`; similarity `k=8`; MMR `k=8, fetch_k=20, lambda_mult=0.7`
Reranker	cross-encoder top_n=3; cross-encoder top_n=5
Vector store	FAISS
Embeddings	TritonAI/OpenAI-compatible embedding model
Generator	TritonAI/OpenAI-compatible chat model

This gives 2 x 3 x 2 = 12 meaningfully distinct RapidFire configurations, satisfying the requirement to demonstrate at least 8 configs.

The RapidFire run command was:

```
python3 rapidfire_experiments.py --input data/golden_qa.json --experiments-path rapidfire_ex
```

The run artifacts are:

- `logs/rapidfire.log`
- `rapidfire_experiments/run_evals_return.json`
- `rapidfire_experiments/mlflow_metrics_summary.csv`
- `rapidfire_experiments/mlflow_runs_full.csv`

4. Results and Analysis

RapidFire evaluated 12 RAG configurations on the 25 curated golden questions. The best run by file-level F1 was pipeline 7:

Pipeline	Chunk size	Overlap	Search	k	File precision	File recall	File F1	Answer nonempty
7	512	96	similarity	5	0.7067	1.0000	0.8000	1.0000

Pipeline	Chunk size	Overlap	Search	k	File precision	File recall	File F1	Answer nonempty
9	512	96	similarity	8	0.7000	1.0000	0.7933	1.0000
1	320	64	similarity	5	0.6667	1.0000	0.7733	1.0000
3	320	64	similarity	8	0.6600	1.0000	0.7667	1.0000
11	512	96	MMR	8	0.6533	1.0000	0.7600	1.0000

Similarity search performed better than MMR on this curated set. The larger 512-token chunk with 96 overlap gave the best top run, while increasing `k` from 5 to 8 slightly reduced precision in the best chunking setup. This suggests that the documentation questions usually needed one or a few focused chunks, and adding more retrieved material increased extra source files without improving recall.

The final `main.py` setting is `c09_title_related_420_80_k1`: 420-token chunks, 80-token overlap, similarity retrieval, top-1 primary chunk, title/lexical reranking, expanded source spans, and related-source expansion. This is intentionally conservative for submission: it keeps the output context under the 2,000-token budget while keeping source spans broad enough for the released evaluator's line-overlap retrieval metric.

Final rerun metrics:

Dataset	Retrieval F1@5	Retrieval score	Correctness	Faithfulness	Completeness	Partial generation score
Released validation/dev set	1.0000	1.0000	0.6444	0.9556	2.33 / 5	0.6889
Curated golden set	0.8400	0.8400	0.6000	0.9600	2.28 / 5	0.6720

The validation judge run had 2 judge-call failures after retries; the JSON report was still written to `outputs/validation_generation_metrics.json`. The curated judge run completed with 0 judge failures.

5. AI Tools Statement

GPT was used to help organize commands, revise the report structure, summarize project requirements, and check consistency between the implementation

and the project statement. The implementation choices, data files, commands, and reported metrics should be verified from the project files and generated run artifacts.